Source-Bound Validation and Contract-Indexed Accounting for Constant-Shape LLM Workflows

Nambi Rajan M[a], Ranjithkumar S[a]

[a] *Vellore Institute of Technology, Vellore, India*

Abstract

Hiding prompt contents does not hide a workflow's model identities, token counts or instrumented tool events. OrchLang is a bounded language whose compiler aligns secret paths, pads model usage and completes loops with cover events. An executable Lean validator establishes conditional usage-trace noninterference. Source-bound acceptance prevents a proposed repair from choosing its own original program or secrecy policy. A compositional certificate establishes trajectory admission from explicit value, tokenizer, response and tool contracts. We implement a byte-bounded adapter and check concrete candidates for twelve workflow variants. The Python frontend and native serving code remain trusted. Independent interpreters agree on 6,320 finite audit executions. A full study of 1684 real-model pairs on one frozen implementation preserves all saved fields and gives one shaped logical trace per workflow. Main-study median weighted logical overhead is 1.84 times original usage. A separate single-batch native-prefill prototype fixes the batch-size projection, but sequence metadata still reveals request length and one batching configuration fails visible-output preservation. The contribution is an executable integration of source-bound validation, contract-indexed accounting and admission for bounded LLM workflows. Timing protection, provider billing equivalence and physical failure noninterference require additional serving contracts.

Keywords: workflow languages, information flow, resource contracts, translation validation, Lean, LLM systems

Email addresses: nambirajan.m2024@vitstudent.ac.in (Nambi Rajan M), ranjithkumar.s@vit.ac.in (Ranjithkumar S)

1. Introduction

An LLM application can hide every prompt from its dashboard and still reveal which path it took. A support workflow may classify a private message, call a specialist on one route, and repeat generation on another. Ordered model identities and token counts can reveal the route without revealing the message. Token-count and routing-metadata attacks already establish this threat [1, 2, 3]. The programming problem is how to control the usage of an entire bounded workflow while preserving the results on which later decisions depend.

Constant-resource programming, control-flow balancing, padding and translation validation are established techniques [4, 5, 6, 7]. Applying them to LLM workflows introduces an execution-contract problem. An operation includes a request, tokenizer, chat template, decoding caps and visible-response rule. Two operations with equal caps need not select equal response kernels, and token counts in one model's unit do not bound another model's requests. A validator's guarantee also depends on the original program and secrecy declarations against which it checks the repair.

OrchLang makes these boundaries explicit. Its untrusted shaper proposes padding, dummy calls and bounded-loop completion. A trusted reader independently derives the original core and declared secrets from source; Lean checks erasure, labels, randomness sites and caps. Execution counts each complete request before dispatch, checks returned usage and retains generated token IDs and request hashes. A per-model reservation ledger refuses unsupported budgets and retains reservations when dispatch usage cannot be confirmed.

The system studied here consists of the Python frontend and runtime and the Lean core validator. Source-bound checking connects these components: the compiler may propose a repair, but it cannot choose the original or the secrecy policy against which that repair is checked. Earlier certificate experiments are retained as historical supplementary material rather than components of this architecture.

OrchLang is an offline programming and assurance tool for a declared logical reporting contract. A workflow author can validate a proposed repair against the actual source and inspect its model-indexed envelope. The author can then assess whether a particular serving interface supports that contract. Deployment confidentiality additionally requires an independently enforced observer and a supported admission domain.

The paper makes three contributions. First, source-bound acceptance connects declared secrets and useful requests to an executable translation validator. Second, model-indexed accounting and a compositional admission certificate connect fixed shapes to resource envelopes under explicit primitive contracts. Third, independent operational comparisons and real-model records test the implementation and expose its limits. These contributions adapt established resource and translation-validation methods to bounded LLM workflows.

2. Observation and scope

The observer receives an ordered list of events

$$e ::= \mathsf{llm}(m, t_{\text{in}}, t_{\text{out}}) \mid \mathsf{tool}(f).$$

Here m denotes a model contract and f a tool name. Contents, site IDs, dummy flags, natural stopping lengths and private audit records are excluded. Tool events are logical instrumentation: a dummy tool event does not invoke a real external service. An external tool provider or packet observer therefore requires a different execution contract.

The prototype's event is an accounting record produced by a trusted wrapper. In isolated-padding mode, that wrapper aggregates a filler prefill and an unchanged request prefill into one input count. An observer below the wrapper can distinguish those submissions and recover the actual request length. The implementation therefore evaluates fixed resource envelopes and budget reservations; it does not enforce input-length confidentiality at the model server. For an observer restricted to exported logs, permitted report normalization gives the same trace abstraction without dummy inference. Our normalization baseline is consequently a practical alternative, not an inadmissible defense.

Executing cover work remains useful for testing whether an envelope can actually be consumed and reconciled against local records. That engineering purpose does not itself provide a security advantage over normalization. A serving interface that enforces a fixed external trace would require a separately specified adapter, including its batch metadata, cache behavior, stopping policy and failures.

The model and trusted host may see secret contents. The theorem concerns a separate usage observer and excludes elapsed time, streaming, pack-

Table 1: Scientific delta and inherited mechanisms. The final column identifies the additional contract studied here, not a claim that prior tools cannot be extended.

Prior method	Established contribution	OrchLang's specialized integration
Agat, constant-resource types, FaCT [4, 5, 8]	Balance control paths and constrain resource-dependent observations.	Bind useful model requests, visible caps and cover work to separately identified model contracts.
Translation validation [7, 9]	Check a proposed translation against a supplied original.	Re-read source bytes and declared secrets independently of the candidate; reject omitted-policy and substituted-original artifacts.
Supersequence defenses, Pacer [6, 10]	Shape traffic and provide cover events under a specific observer.	Align ordered model/tool events and retain useful-call order; local tool dummies do not establish network or external-tool equivalence.
RelCost, eRHL [11, 12]	General relational resource or probabilistic reasoning.	Executable validation and a direct fixed-shape budget corollary, rather than a new logic.
SICC and budget tools [13, 14, 15]	Policy-relative communication compilation or resource admission.	Connect source-bound model/token/stop contracts to exact logical traces, per-model reservations and inspectable execution evidence.

Interface	Assurance and evidence
Checked core	Conditional Lean theorems for ordered logical usage and useful stores.
Admission specification	Equal completion/refusal observations require uniform admission; a domain certificate covers every advertised input and tape.
Python/local model	Finite request, token-ID and execution checks; trusted implementation.
Gateway, tool provider, clock	No physical-observer refinement; logical dummy tools, isolated prefill and EOS conventions can differ at these interfaces.

Figure 1: Threat-model and confidence boundaries. Each lower interface needs additional evidence or assumptions; passing a higher-level check does not certify it.

ets, shared caches, failures, concurrency and contents. A retained latency-augmented attack illustrates this boundary: it reaches 100 percent balanced accuracy in one workflow. Constant logical usage does not imply constant running time.

For stores $\sigma \equiv_{\mathsf{L}} \tau$ agreeing on public variables, usage noninterference requires equal traces under each fixed randomness tape. Public inputs may select different traces. A globally constant shape is a stronger case used by the evaluated secret-input workflows.

The *source contract* binds exact bytes, original core, slots and declared secrets. The *model contract* binds artifact hash, template/tokenizer family, stop policy, optional decoding grammar and padding mode. The *execution contract* defines admission, logical accounting, forced output and visible prefix. Scalar source token annotations help size targets; they are not sound bounds transferred between tokenizers. Before dispatch, the host counts the fully rendered request in the receiving model's unit and refuses overflow instead of silently shortening it. A refusal may itself reveal information. Protection including failures requires a supported admission domain and compatible failure protocol. Section 4.3 proves both a restricted-domain result and a compositional sufficient admission condition. Section 6.4 implements guarded byte bounds for the pinned local adapters. Native correspondence and a physical failure refinement remain separate obligations.

3. Language and shaping

3.1. Bounded programs

The source language has labelled text inputs, model profiles, templated prompts, capped calls, pure tools, conditionals and bounded loops. For example:

```
workflow Router {
  input q: text secret max_tokens 48;
  model m = "llama1b";
  prompt classify(x) = user "Classify: {x}";
  prompt respond(x) = user "Answer: {x}";
  let route = call classify(q) using m max_tokens 3;
  if route == "short" {
    let answer = call respond(q) using m max_tokens 16;
  } else {
    let answer = call respond(q) using m max_tokens 64;
  }
  output answer;
}
```

Unshaped usage exposes request lengths and the selected cap. Shaping fixes the classification usage and aligns the response arms while retaining their visible caps. The core contains sequencing, calls, tools, dummies, conditionals and loops. Expressions contain literals, variables, concatenation, tool-argument escaping and template-level message trimming. Argument escaping prevents separator characters or backslashes from changing argument boundaries and is represented in Lean as well as Python.

Labels form $\mathsf{L} \sqsubseteq \mathsf{H}$. Flow-insensitive propagation joins argument and program-counter labels. A high call requires fixed usage, a high conditional requires equal arm shapes, and a high loop requires a shaped body completed to its public bound. Model responses inherit the confidentiality of their requests. Figure 2 gives the complete core grammar and its relation to the surface syntax. The rules below use the same constructors as the executable Lean definitions.

3.2. Targets, padding and output

The compiler sizes targets from tokenized literals, variable caps and slack. This is an execution-envelope heuristic. Subword concatenation and cross-model values invalidate a general scalar-bound claim. An audit example

$$
\begin{array}{rcl}
\ell & ::= & \mathsf{L} \mid \mathsf{H} \qquad (\mathsf{L} \sqsubseteq \mathsf{H}) \\
e & ::= & s \mid x \mid e \mathbin{+\!\!+} e \mid \mathsf{escape}(e) \mid \mathsf{trim}(e) \\
g & ::= & x = s \mid \mathsf{prefix}(x, s) \\
\pi & ::= & \bot \mid (T, N) \\
p & ::= & \mathsf{skip} \mid p; p \\
 & & \mid \mathsf{call}_i(x, m, k, e, \pi) \mid \mathsf{tool}(x, f, e) \\
 & & \mid \mathsf{dummy}(m, T, N) \mid \mathsf{dummyTool}(f) \\
 & & \mid \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ p \\
 & & \mid \mathsf{loop}(n, g, p, b), \qquad b \in \{\mathsf{false}, \mathsf{true}\}.
\end{array}
$$

Strings are s; variables and randomness sites are natural numbers x, i. The model contract is m, the useful output cap is k, and padding annotation (T, N) fixes logical input and output counts. The loop runs at most n times, stops when g holds, and emits cover iterations when $b = \mathsf{true}$. This grammar lists every constructor of Lean's `Expr`, `Guard` and `Stmt`; $\pi = \bot$ denotes an unpadded call.
The corresponding surface fragments are

```
input x: text secret max_tokens c;
prompt P(x) = user "template {x}";
let y = call P(x) using m max_tokens k;
let y = tool f(x);
if x == "value" { ... } else { ... }
repeat n until x == "done" { ... }
```

The trusted frontend expands templates into expressions, allocates slots and sites, and joins labels into a fixed context Γ. Literal model/template framing is part of the request expression rather than an implicit token charge.

Figure 2: Complete core grammar and representative surface constructs. Surface parsing and lowering are tested implementations outside the Lean proof.

used a string counted as one BERT token whose complete GPT-2 request required 259 tokens against an 18-token target. Exact complete-request admission addresses that boundary. The earlier exact-tokenization study is summarized in the historical supplement; neither observation proves arbitrary static tokenizer bounds.

Default isolated padding prefills filler in a separate context, resets it and evaluates the unchanged request. Logical input usage sums filler and request counts. This is an accounting stress test: two submissions remain distinguishable below the wrapper, and neither single-call input hiding nor equal compute time follows from their sum. In-band filler changes the visible prompt and may change responses. Legacy truncation remains explicitly selectable for reproduction but is not the default publication configuration.

Forced decoding generates N token IDs, continuing past a stop token when needed. The workflow receives the prefix before the first stop, cut to its original cap $k \leq N$. The logical output unit is the returned ID list: natural calls exclude the terminating stop ID, while forced calls count all N IDs, including stop and post-stop IDs. Re-encoding decoded text is not an equivalent count, and provider EOS-accounting conventions may differ. Under fixed serving behavior, isolated padding and prefix extraction are intended to preserve the natural response; this remains an explicit assumption and an empirical check. The implementation recognizes EOS and declared single-token stop IDs. It rejects empty or multi-token stop declarations before workflow execution, because string-level matching across a visible-cap boundary is outside its semantics.

3.3. Alignment and loops

A two-sequence dynamic program aligns compatible model contracts or equal tool names, merges counts by componentwise maximum and inserts missing dummy events. For the evaluation weight $t_{\text{in}} + 4t_{\text{out}}$, it minimizes the common two-arm profile cost. Each contract receives input weight one and output weight four for this scalar score. This is a chosen decode-sensitive objective, not a tokenizer conversion or measured price. Section 6.3 varies the output weight while holding workflows and model contracts fixed. Multi-arm alignment is progressive and heuristic; nested shaped constructs are atomic. Global optimality is not claimed. These mechanisms adapt balancing and supersequence defenses [4, 6].

After early loop termination, ghost iterations emit the body's shape without updating the store or reading real-site randomness. Model dummies

perform real local generation. Dummy tool events only add instrumented records, avoiding external effects. The loop emits its shape repeated to its declared bound. Figure 3 makes the store and randomness behavior explicit.

4. Validation and mechanized properties

4.1. Acceptance from a bound source

Translation validation checks each proposed transformation rather than trusting the optimizer [7, 9]. The source reader derives the original, declared secrets, slots and labels independently of candidate code. A candidate cannot replace the original or omit secrets by supplying a weaker list. The JSON interface requires an explicit list and checks that every listed input is high. Because a hash identifies bytes without proving their interpretation, the Python parser, lowerer and serializer remain trusted.

A small end-to-end negative case makes the source binding consequential. For a source-declared secret query, a candidate supplies an empty secret list and an unshaped natural call. The core checker accepts that call under the weakened all-public policy. The source-bound entry point re-derives the secret declaration and rejects the same call; it accepts the compiler's padded repair. The executable `experiments/source_binding_case.py` retains all three actual Lean verdicts. This is a policy-omission attack on an unbound interface, not an unsoundness in the core validator under its supplied policy.

Lean checks the padding discipline, separated high/public randomness sites, visible caps no larger than forced lengths, and equality of normalized erasures. Erasure removes padding, dummies and loop-completion flags. Useful requests cannot be changed, reordered or dropped. Only SHA-pinned aliases with identical template, stop, decoding-grammar and padding contracts share a model identity. Experiments verify weight bytes. Figure 4 states the label and acceptance rules. The source-bound rule separates the trusted reader's responsibility from the checked core premises.

4.2. World and hypotheses

Execution is parameterized by a world w containing tokenization, natural and forced generation, padding, visible-prefix extraction and pure tools. A tape $\omega(i, j)$ supplies randomness to the jth execution of site i. Dummies and ghost iterations do not advance real-site counters. Pointwise equality induces equal trace laws for a common secret-independent tape distribution. Lean

Write $\langle p, \sigma, c\rangle \Downarrow_{w,\omega} \langle \sigma', c', t\rangle$ for the store, site counters and emitted trace. Let $r = [\![e]\!]_\sigma$ and $a = \omega(i, c(i))$. The rules below abbreviate the recursive function `exec`.

$$\frac{y = \mathsf{gen}_m(r, k, a)}{\langle \mathsf{call}_i(x, m, k, e, \bot), \sigma, c\rangle \Downarrow \langle \sigma[x \mapsto y], c[i \mapsto c(i)+1], [\mathsf{llm}(m, \mathsf{tok}_m(r), \mathsf{outTok}_m(y))]\rangle}$$

$$\frac{r' = \mathsf{padReq}_m(T, r) \quad y = \mathsf{forced}_m(r', a, N)}{\langle \mathsf{call}_i(x, m, k, e, (T, N)), \sigma, c\rangle \Downarrow \langle \sigma[x \mapsto \mathsf{view}_m(k, y)], c[i \mapsto c(i)+1], [\mathsf{llm}(m, \mathsf{tok}_m(r'), \mathsf{outTok}_m(y))]\rangle}$$

$$\frac{\langle p, \sigma, c\rangle \Downarrow \langle \sigma_1, c_1, t_1\rangle \quad \langle q, \sigma_1, c_1\rangle \Downarrow \langle \sigma_2, c_2, t_2\rangle}{\langle p; q, \sigma, c\rangle \Downarrow \langle \sigma_2, c_2, t_1 +\!\!+ t_2\rangle}$$

$$\frac{[\![g]\!]_\sigma = \mathsf{true} \quad \langle p, \sigma, c\rangle \Downarrow \langle \sigma', c', t\rangle}{\langle \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ q, \sigma, c\rangle \Downarrow \langle \sigma', c', t\rangle}$$

The false-guard conditional rule selects q. A tool writes $\mathsf{toolFn}_f(r)$ to x, leaves counters unchanged and emits $[\mathsf{tool}(f)]$. A model dummy emits its padded/forced event at cell 0 without updating the store or counters; a tool dummy emits only $[\mathsf{tool}(f)]$.

$$\frac{\neg[\![g]\!]_\sigma \quad \langle p, \sigma, c\rangle \Downarrow \langle \sigma_1, c_1, t_1\rangle \quad \langle \mathsf{loop}(n, g, p, b), \sigma_1, c_1\rangle \Downarrow \langle \sigma_2, c_2, t_2\rangle}{\langle \mathsf{loop}(n+1, g, p, b), \sigma, c\rangle \Downarrow \langle \sigma_2, c_2, t_1 +\!\!+ t_2\rangle}$$

A zero-bound loop emits []. If its guard already holds, an unpadded loop emits []; a padded loop emits n copies of the body's ghost trace and leaves the store and counters unchanged. Under conformance on the cover domain, that ghost trace equals $\mathsf{shape}(p)$. These rules include no physical exceptions.

Figure 3: Operational semantics, with the fixed world and tape omitted from rule arrows for readability. An initial accumulated trace is prepended to t.

Let $L_\Gamma(e)$ join variable labels, and let $\ell = pc \sqcup L_\Gamma(e)$ and $pc' = pc \sqcup \Gamma(\mathsf{guardVar}(g))$. The judgment $\Gamma; pc \vdash p$ abbreviates `wt` returning true.

$$\frac{\ell \sqsubseteq \Gamma(x) \quad (\ell = \mathsf{H} \Rightarrow \pi \neq \bot)}{\Gamma; pc \vdash \mathsf{call}_i(x, m, k, e, \pi)} \qquad \frac{pc \sqcup L_\Gamma(e) \sqsubseteq \Gamma(x)}{\Gamma; pc \vdash \mathsf{tool}(x, f, e)}$$

$$\frac{\Gamma; pc' \vdash p \quad \Gamma; pc' \vdash q \quad pc' = \mathsf{H} \Rightarrow \mathsf{shape}(p) = \mathsf{shape}(q) \neq \bot}{\Gamma; pc \vdash \mathsf{if}\ g\ \mathsf{then}\ p\ \mathsf{else}\ q}$$

$$\frac{\Gamma; pc' \vdash p \quad pc' = \mathsf{H} \Rightarrow (b = \mathsf{true} \wedge \mathsf{shape}(p) \neq \bot)}{\Gamma; pc \vdash \mathsf{loop}(n, g, p, b)}$$

Skip and dummies are well typed; sequencing requires both components to be well typed. Separate structural checks require $k \leq N$ at padded calls and mark sites reached under a high program counter. Let H be that site set.

$$\frac{\Gamma; \mathsf{L} \vdash p \quad \mathsf{sitesOK}(\Gamma, H, \mathsf{L}, p) \quad \mathsf{capsOK}(p) \quad \mathsf{clean}(\lfloor p \rfloor) = \mathsf{clean}(\lfloor p_0 \rfloor)}{\mathsf{validateProgram}(\Gamma, p_0, p) = \mathsf{true}}$$

Erasure removes padded annotations and dummies, preserves useful calls/tools and control flow, and clears loop-completion flags. Normalization `clean` removes skip/sequence artifacts without changing useful order.

$$\frac{\mathsf{readSource}(b) = (p_0, \Gamma, S) \quad \forall x \in S,\ \Gamma(x) = \mathsf{H} \quad \mathsf{validateProgram}(\Gamma, p_0, p) = \mathsf{true}}{\mathsf{acceptSource}(b, p)}$$

The last rule describes the trusted Python entry point. Its source reader, unlike the candidate, supplies the original and the declared secret set S. Lean checks the core premises, but does not prove that `readSource` implements parsing.

Figure 4: Label propagation, padding discipline, erasure and source-bound acceptance. The displayed rules match the executable checks rather than adding a second type system.

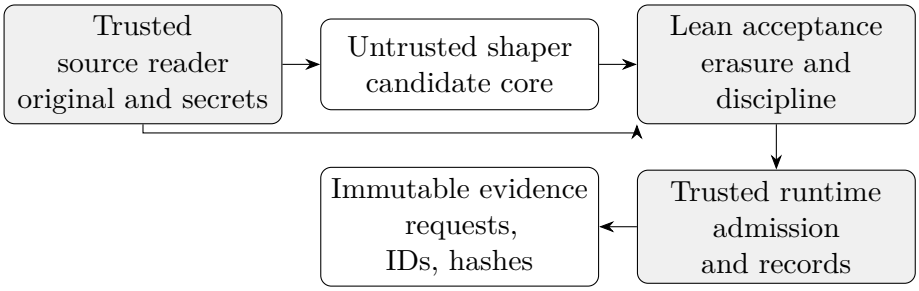


Figure 5: Shaded components remain trusted. Lean proves properties of the checked core, not the complete pipeline. The source reader supplies the original and secret declarations directly to acceptance.

includes the direct finite weighted-list form, not a measure-theoretic library or a verified physical random generator.

Two explicit propositions state the primitive assumptions:

$$\begin{aligned} \mathsf{Conforms}(w): \quad & \mathrm{tok}_m(\mathrm{padReq}_m(T,r)) = T, \\ & \mathrm{outTok}_m(\mathrm{forced}_m(r,c,N)) = N; \\ \mathsf{Transparent}(w): \quad & \mathrm{view}_m(k, \mathrm{forced}_m(\mathrm{padReq}_m(T,r),c,N)) \\ & \qquad = \mathrm{gen}_m(r,k,c) \qquad (k \le N). \end{aligned}$$

These quantify over the abstract world's arguments. The runtime is partial and rejects unsupported requests; a total refinement into this world is not proved. A post-dispatch count check detects a violation but cannot retract exported usage.

4.3. Conformance on an admitted domain

Global conformance is stronger than a nontruncating padder can offer: an arbitrary target may be smaller than the request. The separate development `proof/AdmissionDomain.lean` replaces that premise with $\mathsf{ConformsOn}(w, D)$. Here $D(m, T, N, r)$ identifies supported request contracts; the two conformance equations are required only for members of D, for every randomness cell. For example, a concrete domain must exclude oversized requests and unsupported context/output limits.

The recursively defined $\mathsf{RunsIn}(w, \omega, D, p, s)$ checks membership for every padded call actually reached, including model dummies and loop-completion calls. It follows evaluated intermediate stores and loop guards. It does

not assume that scalar source input caps imply membership. The observation alphabet adds refused, completed(trace), failed(prefix) and unknown-dispatch(prefix). An ideal admission adapter emits completed core execution exactly when RunsIn holds, and refused otherwise. This is a noncomputable specification, not a physical preflight algorithm that predicts future responses without inference.

Theorem 1 (Restricted-domain shape and admission)**.** *If* $\mathrm{shape}(p) = s$ *and* w *conforms on* D*, each execution satisfying RunsIn emits exactly* s*. Two initially empty-trace executions have equal ideal completion/refusal observations if their RunsIn predicates are equivalent. A domain certificate establishing RunsIn for every advertised input and every tape makes every such observation completed(*s*).*

The declarations `shape_exec_on`, `admitted_noninterference` and `certified_domain_noninterference` prove these statements by structural and loop induction. They require neither global conformance nor transparency. The domain certificate is an explicit proof obligation. RunsIn itself is a trajectory predicate, so evaluating it is not a preflight algorithm for a chain whose later prompts depend on generated text. In the physical runtime, a first call can complete before a later rendered request exceeds its target. One run then exposes failed($[e_1]$), while another completes with $[e_1, e_2]$. Per-call refusal prevents overspending at the next dispatch but does not hide this prefix-length channel. The sufficient certificate below addresses request admission; transport failures and unknown dispatches remain outside its claim.

An executable falsification uses two public-equal stores: a short secret completes, while an oversized secret is refused before dispatch. Thus resource-safe rejection alone does not prove failure-observation noninterference. A separate exhaustive check covers the 15 strings over $\{a, b\}$ of length at most three, under three seeds in the character-token mock world: all 45 executions complete with the static shape. This finite check illustrates premise checking; it does not certify arbitrary strings, model trajectories or stochastic tapes.

4.4. Compositional static admission

An admission certificate can avoid lookahead by bounding the values from which every request is constructed. Let $|r|_b$ measure text size in a declared unit, such as UTF-8 bytes, and let the finite vector v bound every store slot.

The invariant is $I_v(\sigma) \equiv \forall x, |\sigma(x)|_b \le v_x$, with unlisted slots bounded by zero. Define the computable expression envelope

$$\begin{aligned} E_v(s) &= |s|_b, & E_v(x) &= v_x, \\ E_v(e_1 +\!\!+ e_2) &= E_v(e_1) + E_v(e_2), & E_v(\mathsf{escape}(e)) &= 2E_v(e), \\ E_v(\mathsf{trim}(e)) &= E_v(e). \end{aligned}$$

The envelope contract requires subadditive concatenation, at most twofold escaping and nonexpansive trimming. It also supplies receiving-model token bounds $\mathsf{tok}_m(r) \le \alpha_m|r|_b + \delta_m$, visible-response bounds $|\mathsf{view}_m(k, \mathsf{forced}_m(\mathsf{padReq}_m(T, r), a, N))|_b \le \rho_m(k)$ for $k \le N$, and tool-result bounds θ_f. These are primitive assumptions with a common text-size unit, not conversions between two models' token counts. A concrete implementation must justify them.

For each padded call, the checker tests

$$\alpha_m E_v(e) + \delta_m \le T, \qquad T + N \le C_m, \qquad k \le N, \qquad \rho_m(k) \le v_x.$$

Tool assignments require $\theta_f \le v_x$. Dummies are checked against their actual fixed request and context limit. Both conditional arms and sequence components must pass. Loops require a certified body and, when padded, a certified cover profile. No call to `exec` or RunsIn occurs in this checker. An input guard checks I_v before the first dispatch; the proof preserves this invariant through every useful assignment and leaves it unchanged under cover work.

Theorem 2 (Static certificate implies trajectory admission)**.** *Let the envelope contract hold for w, and let the compositional checker accept p under v. For every initial store satisfying I_v and every tape ω, $\mathsf{RunsIn}(w, \omega, D_b, p, s)$ holds, where $D_b(m, T, N, r) \equiv \mathsf{tok}_m(r) \le T \wedge T + N \le C_m$. The final store also satisfies I_v.*

Store preservation is proved by `static_certificate_sound`. The theorem `static_certificate_implies_RunsIn` establishes admission by statement and loop induction. `static_certificate_domain` supplies the premise of the restricted-domain theorem for all inputs satisfying the guard. With conformance on D_b and a fixed shape, every admitted core execution therefore completes with that shape. This result removes the trajectory-lookahead obligation for certified envelopes; it does not assume a prediction of future model text.

The optional byte-bounded frontend implements this sufficient checker on source-derived candidates. It guards input buffers, bounds tool results and applies a declared UTF-8 response buffer after decoding. Targets use E_v rather than a heuristic conversion between token units. Lean's actual `staticCertificate` checks the exported candidate and finite bounds. The source-bound validator separately checks erasure and secrecy. Section 6.4 reports all twelve application candidates and their execution checks. The ordinary frontend retains its heuristic mode. The bounded mode is a separate execution contract applied to both original and shaped programs; it is not a claim that every unbounded original fits the chosen buffers. Native tokenization, decoder correspondence and physical failure behavior remain trusted obligations.

Theorem 3 (Core usage noninterference)**.** *Under Conformance, a program accepted by the padding and randomness-site checks has equal usage traces from public-equal stores for every tape.*

The proof inducts over statements. Public calls use equal requests and tape cells; high calls emit fixed usage; high branches emit a common shape; public branches select the same arm. Loop induction handles real and ghost iterations. Confinement preserves public stores and public-site counters under a high program counter.

Theorem 4 (Core functional preservation)**.** *Under Transparency and the cap check, a padded program and its normalized erasure end in the same store for every initial store and tape. An accepted candidate therefore preserves the corresponding unpadded original's store.*

The declarations `validateProgram_sound` and `secret_inputs_sound` combine these checks with declared-secret labelling. Neither property implies task accuracy or verifies the source reader.

4.5. Model-indexed budgets

Let $U_m(s)$ be the triple of input-token sum, generated-token sum and call count in trace s for contract m. Tool events contribute zero to this triple. Different model units remain separate coordinates.

Theorem 5 (Shape accounting and budget admission)**.** *If* $\mathrm{shape}(p) = s$ *and* w *conforms, then every store and tape give* $U_m(\mathrm{trace}(\mathrm{exec}(p))) = U_m(s)$*. If* $U_m(s) \le B_m$ *componentwise, execution stays within* B_m*.*

The declarations `shape_usage_exact` and `shape_budget_sound` are direct corollaries of `shape_exec`, not a new cost logic. The runtime admits a complete shape against budgets before inspecting inputs. Without a complete shape, only per-call reservations apply. The Python ledger is tested, not a mechanized refinement or concurrent ownership system.

Accounting alone cannot replace the ordered shape check. For distinct fixed events A and B, paths $[A, B]$ and $[B, A]$ have the same per-contract totals but different observations. A common profile such as $[A, B, A]$ preserves each path's useful order by filling its missing position with a dummy. Conversely, equal caps cannot justify replacing a request with different text: the response kernel may change even when usage does not. The erasure check and model contract therefore complement the budget theorem rather than following from it.

Lean 4.19.0 checks the development without admissions. Noninterference and validator axiom reports contain only `propext` and `Quot.sound`; the budget corollaries require only `propext`. JSON decoding correctness, parsing, concrete tokenizers and backend code remain outside the formal boundary. The domain extension's shape proof also uses only `propext`; the ideal admission results additionally use Lean's classical choice and quotient axioms. Neither development uses a `sorryAx` placeholder.

5. Implementation and custody

The runtime admits complete requests, reserves usage before dispatch and confirms counts/caps on return. Transport exceptions or nonconforming returns leave usage incomplete, with reservations retained rather than recorded as zero-cost success. Private request/token evidence supports inspection without entering classifier features. Each manifest binds source, dataset, selected IDs, implementation hashes, model contracts, decoding parameters, platform, threads and optional backend-library hashes. Resume requires the same manifest and rejects duplicate or foreign IDs. Unbound historical records cannot be silently resumed under the new runtime.

A separate `python -m orchlang check` command re-reads source and model bytes to check an exported artifact. It does not trust the artifact's original or secret declarations. Request hashes and token-ID records allow separate checks of the declared model contract and the wrapper's aggregate counts. These records complement the proof: they can expose a concrete

disagreement without establishing correctness for every tokenizer input or physical execution.

6. Evaluation

The evaluation separates four questions: whether the implementations agree on the checked core, whether one frozen implementation preserves application outputs, whether source-derived byte envelopes pass the admission certificate, and which native observations survive a single-batch serving prototype. All inference is local. The full application study and the three additional experiments below have separate protocols and denominators.

6.1. Core checks and implementation identity

All 47 configured tests pass, including source binding, immutable resumption, model-indexed reservations, unsupported stop declarations, complete token-piece decoding and pre-dispatch byte guards. Actual pinned GGUFs enable the tokenizer and template tests; model-free runs report those cases as skips. Generated differential testing retains 1,000 authored programs, 5,000 original/candidate Lean checks and 16,000 trace/store comparisons without a counterexample. A separate operational comparison covers 150 programs in four contexts under two finite kernels and five variants, plus a targeted template program. Python, an independent interpreter and Lean's actual `exec` agree on 6,320 executions and 604 validator checks. These audit worlds use character counts. They test implementation agreement without verifying the frontend or native tokenizer. Retained fault injection rejects all 103 mutants that leak under its finite oracle, among 210 mutants over 12 operators.

Before application inference, we froze production code at `41a7d81` and ran from a clean detached checkout. All nine main workflows and three Qwen variants use that identity. Manifests bind the complete implementation, source, data, selected IDs, model contracts and native-library hashes. The study uses Python 3.12.14, llama-cpp-python 0.3.36, native build 11490 (`9c2e0e491`), four CPU threads, a 4,096-token context, batch size 128 and greedy decoding. GPU execution uses Vulkan on an NVIDIA RTX 5050 Laptop GPU, with 99 requested offload layers. Seeds are fixed by dataset index and useful-call site. We report logical usage; this single-machine protocol supports no throughput or timing-protection claim.

Table 2: One frozen implementation. K is the observed decision-class count; original balanced accuracy (BA) and task accuracy are percentages. Tr. and MI are distinct shaped traces and empirical trace/decision mutual information in bits. Cost is shaped/original weighted logical usage. Q denotes Qwen replication; a dash omits an unavailable task scorer or an uninformative single-class attack.

Workflow	Pairs	K	Orig BA	Tr.	MI	Cost	Task acc.
Routing	150	2	95.4	1	0	1.84	65.3
Chaining	150	2	100.0	1	0	6.10	–
Evaluator	100	3	100.0	1	0	2.01	–
Orchestrator	100	3	100.0	1	0	1.13	–
PF switch	210	4	26.5	1	0	1.54	37.6
Email	152	4	59.7	1	0	1.66	–
Corrective RAG	150	2	75.0	1	0	1.75	34.0
Calculator	100	3	100.0	1	0	13.99	2.0
Adaptive RAG	60	6	83.3	1	0	2.29	53.3
PF switch (Q)	210	4	55.8	1	0	5.49	58.6
Email (Q)	152	4	74.0	1	0	2.13	–
Corrective RAG (Q)	150	1	–	1	0	1.81	38.0

6.2. Full homogeneous application study

The main study contains 1,172 paired inputs over routing, chaining, evaluator–optimizer, orchestrator–worker, switch, email, calculator, corrective RAG and adaptive RAG. It uses Llama 3.2 1B, with Llama 3.2 3B for the adaptive workflow's larger-model path. Replication contains 512 Qwen 2.5 1.5B pairs over switch, email and corrective RAG. The bounded ports retain their declared loop limits, retrieval corpora and primary grading prompts. Failed or mismatched attempts cannot be removed from the completion denominator.

All 1684 pairs complete with equal saved fields, one shaped trace per workflow and no recorded truncation. The independent record audit checks 3,368 execution records and 10698 useful and cover model calls. It re-tokenizes every complete request, reconstructs visible prefixes and reconciles per-model ledgers. It also matches private local eval/yield records to public events, including duplicate multiplicities. A deliberately changed input count is detected. These records support the declared client accounting contract; they are not provider attestations or a proof of every intermediate store value.

Attack features retain ordered model/tool identities, counts, event count and model-call count. A class-balanced random forest uses five-fold cross-

validation, stratified when class counts permit. These are descriptive classifiers on the authored corpus. Original switch prediction is near chance, and the primary Qwen corrective-RAG port has one observed decision class. Neither supplies a general leakage result. Every shaped feature vector is constant within its workflow, giving empirical trace/decision MI zero. Fold-dependent prediction scores can still differ from $1/K$ because the trained constant prediction depends on each fold's labels; this does not reveal information in a constant trace.

Output preservation retains task errors. Main-study routing, switch, calculator, corrective RAG and adaptive RAG achieve 65.3, 37.6, 2.0, 34.0 and 53.3 percent, respectively, using the inherited scorers. Calculator remains a control-flow stress test. Preserving these bounded ports does not recover an upstream application's accuracy or establish independent response quality.

6.3. Cost and observer alternatives

For input weight one and output weight four, main-study median overhead is 1.84 times original usage, with range 1.13–13.99. These weights describe logical usage, not a provider invoice. Static recompilation on these records gives zero median saving against all-branches cover execution, with 52 percent maximum saving; uniform model cells give 16 percent median and 59 percent maximum saving. These are comparisons of validated envelopes, not new executions of commercial defenses. The supplement retains actual alignment/all-branches/uniform/random executions of the separate two-route microbenchmark. Recompilation at output weights 0.25, 1, 2, 4, 8 and 16 accepts all 72 candidates and leaves each of the twelve variants' shapes unchanged. This tests the given profiles rather than establishing weight insensitivity for arbitrary programs.

For an observer restricted to exported reports, normalization achieves the same fixed reporting abstraction with less execution work. Isolated prefill is useful for consuming and auditing an envelope; it adds no input-length protection below the wrapper. Historical latency-augmented attacks reach 100 percent balanced accuracy on one shaped workflow. The present study therefore makes no timing claim. All earlier application cohorts retain their original implementation identities in the supplement and do not enlarge this homogeneous denominator.

6.4. Concrete byte-bounded admission

The optional adapter in Section 4.4 is instantiated for all twelve workflow variants. We scan every vocabulary entry of each pinned GGUF with the complete native token-piece API: the two Llama vocabularies each contain 128,256 entries and Qwen contains 151,936. All three have maximum raw piece length 128 bytes. There are 231 pieces longer than 32 bytes in each Llama vocabulary and 240 in Qwen. The former fixed-buffer wrapper returned an empty piece for a 128-byte witness; the corrected decoder resizes the buffer and retains the complete piece. No generic 32-byte bound is assumed.

The adapter explicitly bounds UTF-8 input buffers, visible responses and tool results. A response buffer has size $128k$ for visible token cap k; it is applied after decoding, because replacement decoding of invalid UTF-8 can expand the byte count. Tool and input buffers use four times their declared token annotation, as a chosen byte policy rather than a tokenizer conversion theorem. Both original and shaped executions use the same buffers. The frontend derives slot bounds from all source assignments and sizes each target with E_v. It then exports the actual candidate and bounds to Lean's `staticCertificate`, alongside independent source-bound validation.

For these byte-BPE tokenizers we declare $\alpha_m = 1$ and $\delta_m = 0$ on valid UTF-8 complete requests, with automatic BOS insertion disabled. Every vocabulary contains all 256 single-byte pieces. The native byte-splitting/merge algorithm remains trusted [16]; 2,058 adversarial and seeded complete requests per model have no counterexample to the bound. Arbitrary isolated non-UTF-8 byte strings are outside the source-string domain. A Lean lemma proves that bounded token-piece lengths and a token cap bound the sum of raw piece bytes; the adapter's post-decoding buffer enforces the actual string bound. The scan and executable certificate do not verify native code or JSON parsing.

The guard admits 1682 of the 1,684 planned inputs; two Calculator inputs exceed the chosen byte policy. The 24 separate pairs complete with equal full runtime stores and their certified shapes. Auditing reconciles 188 local model calls, 36 tool-buffer clips and 0 response-buffer clips. Deliberately oversized inputs are refused before any dispatch for all twelve variants. This is an implemented bounded interface with explicit admission, not evidence that the unbounded primary interface satisfies the same value invariant. The theorem covers all guarded trajectories under its primitive contracts; finite executions test the adapter without proving a physical failure refinement. An earlier 32,768-context attempt completed 16 pairs before device-memory allocation

Table 3: Actual source-derived admission candidates. All twelve pass core and static admission checks at context 21,504. Guard counts refer to the full planned input sets; maximum $T + N$ is taken over the emitted shape. Executed pairs select the shortest and longest admitted UTF-8 input before inference.

Variant	Guard admitted	Max. $T + N$	Equal/shape pairs
PF switch	210/210	1,081	2/2
Routing	150/150	615	2/2
Chaining	150/150	16,782	2/2
Evaluator	100/100	12,771	2/2
Email	152/152	2,234	2/2
Orchestrator	100/100	727	2/2
Calculator	98/100	2,861	2/2
Corrective RAG	150/150	7,293	2/2
Adaptive RAG	60/60	20,606	2/2
PF switch (Q)	210/210	928	2/2
Email (Q)	152/152	2,149	2/2
Corrective RAG (Q)	150/150	7,208	2/2

failed at the adaptive workflow. Those records remain separate: token-context admission alone does not bound simultaneous device allocations.

6.5. Single-batch native prefill

We also implement a separate greedy prototype using one native `llama_decode` submission with 128 tokens. Useful request tokens occupy sequence 0 and filler occupies sequence 1; each sequence has its own positions, and only the last useful token requests logits. Subsequent decode submissions contain one token. Sequence membership provides attention isolation through the native batch interface [17]. This experiment runs on CPU with two model families, three empty/short/multilingual prompt bodies, visible caps 4 and 16, and forced length 24. All attempts remain in Table 4.

For each model, the natural native-size traces have four distinct values and the shaped size projection has one: 128 followed by 23 single-token decode submissions. Normalizing reports above this API cannot change those submissions. Microbatch size 1 preserves all 12 visible outputs. At size 128, one Qwen case differs. Removing the cover cache does not remove that mismatch. These observations refute unqualified transparency for the latter configuration; they do not prove transparency for every input at size 1. Sequence and logit metadata still identify the useful-token subset in every case. The prototype therefore supports a fixed native-size projection, while

Table 4: Native-prefill prototype, 12 attempted cases per configuration. Single counts one exact 128-token prefill; Equal compares the visible output with natural unpadded generation. Metadata counts cases in which sequence IDs recover the useful request length. Cache removal discards filler sequence 1 after prefill.

Native microbatch	Done	Single	Equal	Metadata
128	12	12	11	12
1	12	12	12	12
128 + cache removal	12	12	11	12

a server observing full batch metadata retains request-length information. No claim extends this projection to packets, cache traffic, elapsed time or arbitrary physical failures.

6.6. Categorical corrective-RAG ablation

The primary Qwen grader mixes numeric-score instructions with a yes/no parser. The earlier retained diagnosis found 484 zeros, 34 ones, five yes responses and 77 no responses among 600 grades; accepting numeric ones still sent all 150 questions to web search. We retain that primary port and evaluate a separate contract: a categorical relevance prompt and a grammar permitting only `{"score":"yes"}` or `{"score":"no"}`. The grammar is part of the grader's model-contract identity. Answer generation, retrieval and the rule "search if any of four grades is not yes" remain the same.

The ablation runs all 150 paired questions. All 600 original grades are valid categorical JSON: 62 yes and 538 no. Web search is selected for 149 questions and omitted for 1. Task accuracy is 56.0 percent under the same inherited answer scorer: 84/150 correct, compared with 57/150 for the primary Qwen port. The paired comparison has 28 improved and 1 regressed answers. The route distribution remains strongly imbalanced. All pairs preserve saved fields, including the four grades, and have one shaped logical trace. Format compliance does not establish grader correctness, and this combined prompt/grammar intervention does not identify either component's individual causal contribution.

7. Related work

The program-counter security model formalizes control-flow observations [18]. Verified constant-time compilation preserves an appropriate observation model through compiler passes [19, 20]. OrchLang instead checks

a proposed core translation. Its source reader and physical serving adapter remain trusted, and its observation consists of workflow accounting events.

Agat, constant-resource typing/synthesis and FaCT establish language-based mitigation [4, 5, 8]. RelCost and probabilistic relational logics are more general reasoning frameworks [11, 12]. OrchLang specializes established ideas to bounded model operations; its budget result is a fixed-shape corollary. Supersequence defenses and Pacer establish padding and deployment precedents [6, 10]. Our observation excludes their network/timing setting. Token-count and routing attacks already motivate the threat [1, 2, 3].

Predictive mitigation makes a different tradeoff: delayed releases can bound timing leakage while changing performance [21, 22]. That approach is relevant when a fixed admission envelope cannot be guaranteed. It would require an explicit model of releases, overflows and repeated observations; a time bucket alone does not establish the logical noninterference proved here.

LMQL combines model calls, control flow and output constraints [23]. Its decoding constraints address the language of responses, whereas OrchLang constrains an execution's usage observation. A categorical-output constraint changes the response kernel. Our separate CRAG ablation evaluates that change under its own grammar-indexed contract.

SICC compiles policy-relative communication; communication-graph work studies workflow metadata and cover traffic [13, 24]. Exact logical input/output counts are a scope distinction, not evidence that earlier work ignores workflow structure. CaMeL, FIDES and Hard-KLAIM address agent integrity, information flow, provenance or authority [25, 26, 27]. Hard-KLAIM's adapter assumptions are particularly relevant. Proof-Carrying Contracts likewise distinguishes formal commitments from world correspondence [28]. Authorization and usage noninterference are different properties.

Resource admission overlaps Token Budgets and structural workflow-bound tools [14, 15, 29, 30]. Our sequential ledger is not affine ownership or verified concurrency. Two narrow retained Costwright probes are not a general benchmark. The supplement provides a source-by-source matrix. The literature search does not establish exclusive priority.

8. Limitations and implications

The checker and conditional core theorems are mechanized. Source parsing, serialization, native tokenization and physical serving are trusted. Finite cross-layer agreement tests that boundary without closing the proof gap. The

byte-bounded adapter defines an explicit advertised input domain; refusal outside that guard can reveal information. Successful context checks do not establish device-memory availability, transport success or a physical failure protocol. Side effects, concurrency and stateful tools require richer semantics before entering the theorem's observer.

The application corpus is authored and small. Bounded ports change upstream models, retrieval and loop choices, and poor task outputs remain poor after shaping. A retained census codes 37/54 example applications as content-dependent from 72 listed rows; it is not a population estimate or an independently double-coded study. Single-machine classifiers and paired runs cannot establish deployment security or general usefulness.

The practical artifact lets an author validate a source-bound repair, inspect its per-model envelope and select either heuristic admission or a declared byte-bounded interface. The static certificate removes trajectory lookahead under explicit primitive contracts. It does not verify those native contracts or turn the sequential Python ledger into a concurrent ownership system.

The native-prefill prototype enforces one fixed batch-size projection, but full sequence metadata reveals the real-token subset. Its microbatch-dependent output mismatch also shows why attention isolation alone is insufficient to assume transparency. A deployment needs an observer-specific serving refinement, including metadata, caches, stopping and failures. Paced execution and public loop envelopes would require separate leakage and deadline-miss measurements.

9. Conclusion

OrchLang connects source-declared secrecy to checked workflow shapes, model-indexed accounting and compositional admission. The optional byte-bounded adapter makes the admission certificate executable on actual workflow candidates. A homogeneous study of 1684 real-model pairs preserves saved outputs and fixed logical traces, while independent records expose the conventions behind those counts. The native-prefill experiment fixes a narrower batch-size observer and retains both metadata leakage and a transparency counterexample. These results support an offline programming and assurance artifact whose physical serving guarantees depend on separately justified adapters.

Data and software availability

The accompanying versioned artifact contains source, execution records, protocols, generated tables, dependency versions and SHA-256 integrity manifests. Model weights are obtained separately using the supplied revision and hash pins. The historical repository is https://github.com/guyoverclocked/orchlang-compiler-design-project; the revised sources and evidence are supplied in the accompanying review archive. Reproduction instructions identify the implementation associated with each cohort.

References

[1] R. Weiss, D. Ayzenshteyn, G. Amit, Y. Mirsky, What was your prompt? a remote keylogging attack on AI assistants, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 3367–3384.
URL https://www.usenix.org/conference/usenixsecurity24/presentation/weiss

[2] T. Zhang, G. Saileshwar, D. Lie, Time will tell: Timing side channels via output token count in large language models, arXiv:2412.15431 (2024).

[3] T.-R. Chen, Sensitive-topic leakage through LLM routing metadata: Measurement and mitigation, arXiv:2610.09981 (2026).

[4] J. Agat, Transforming out timing leaks, in: Proceedings of the 27th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), 2000, pp. 40–53. doi:10.1145/325694.325702.

[5] V. C. Ngo, M. Dehesa-Azuara, M. Fredrikson, J. Hoffmann, Verifying and synthesizing constant-resource implementations with types, in: IEEE Symposium on Security and Privacy (S&P), 2017, pp. 710–728. doi:10.1109/SP.2017.53.

[6] T. Wang, X. Cai, R. Nithyanand, R. Johnson, I. Goldberg, Effective attacks and provable defenses for website fingerprinting, in: 23rd USENIX Security Symposium, 2014, pp. 143–157.

[7] A. Pnueli, M. Siegel, E. Singerman, Translation validation, in: Tools and Algorithms for the Construction and Analysis of Systems (TACAS), Vol. 1384 of LNCS, 1998, pp. 151–166. doi:10.1007/BFb0054170.

[8] S. Cauligi, G. Soeller, B. Johannesmeyer, F. Brown, R. S. Wahby, J. Renner, B. Grégoire, G. Barthe, R. Jhala, D. Stefan, FaCT: A DSL for timing-sensitive computation, in: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2019, pp. 174–189. doi:10.1145/3314221.3314605.

[9] G. C. Necula, Translation validation for an optimizing compiler, in: Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation (PLDI), 2000, pp. 83–94. doi:10.1145/349299.349314.

[10] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, D. Garg, Pacer: Comprehensive network side-channel mitigation in the cloud, in: 31st USENIX Security Symposium, 2022, pp. 2819–2838.
URL https://www.usenix.org/conference/usenixsecurity22/presentation/mehta

[11] E. Çiçek, G. Barthe, M. Gaboardi, D. Garg, J. Hoffmann, Relational cost analysis, in: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL), 2017, pp. 316–329. doi:10.1145/3009837.3009858.

[12] M. Avanzini, G. Barthe, D. Davoli, B. Grégoire, A quantitative probabilistic relational hoare logic, Proceedings of the ACM on Programming Languages 9 (POPL) (2025) 1167–1195. doi:10.1145/3704876.

[13] J. Xu, L. Fan, Z. Wang, X. Li, H. Liu, Selection-invariant communication compilers for privacy-aware multi-agent LLM workflows, arXiv:2609.26076 (2026).

[14] S. Khan, Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study, arXiv:2606.04056 (2026).

[15] K. Dickerson, Statically bounding resource accumulation in LLM agent workflows: The cost case, Author-hosted report, https://loom.technology/research/bounded-llm-cost/, 18 May 2026; not peer reviewed (2026).

[16] ggml-org, llama.cpp native vocabulary implementation, revision 9c2e0e491, https://github.com/ggml-org/llama.cpp/blob/9c2e0e491/src/llama-vocab.cpp, accessed 29 October 2026 (2026).

[17] ggml-org, llama.cpp native batch interface, revision 9c2e0e491, https://github.com/ggml-org/llama.cpp/blob/9c2e0e491/include/llama.h, accessed 29 October 2026 (2026).

[18] D. Molnar, M. Piotrowski, D. Schultz, D. Wagner, The program counter security model: Automatic detection and removal of control-flow side channel attacks, in: Information Security and Cryptology (ICISC 2005), Vol. 3935 of LNCS, 2006, pp. 156–168. doi:10.1007/11734727_14.

[19] G. Barthe, S. Blazy, B. Grégoire, R. Hutin, V. Laporte, D. Pichardie, A. Trieu, Formal verification of a constant-time preserving C compiler, Proceedings of the ACM on Programming Languages 4 (POPL) (2020). doi:10.1145/3371075.

[20] G. Barthe, S. Blazy, R. Hutin, D. Pichardie, Secure compilation of constant-resource programs, in: IEEE 34th Computer Security Foundations Symposium (CSF), 2021, pp. 1–12. doi:10.1109/CSF51468.2021.00020.

[21] A. Askarov, D. Zhang, A. C. Myers, Predictive black-box mitigation of timing channels, in: Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS), 2010, pp. 297–307. doi:10.1145/1866307.1866341.

[22] D. Zhang, A. Askarov, A. C. Myers, Language-based control and mitigation of timing channels, in: Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2012, pp. 99–110. doi:10.1145/2254064.2254078.

[23] L. Beurer-Kellner, M. Fischer, M. Vechev, Prompting is programming: A query language for large language models, Proceedings of the ACM on Programming Languages 7 (PLDI) (2023) 1946–1969. doi:10.1145/3591300.

[24] B. Dangol, From privacy to workflow integrity: Communication-graph metadata in autonomous agent interoperability, arXiv:2606.07150 (2026).

[25] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, arXiv:2503.18813 (2025).

[26] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, S. Zanella-Béguelin, Securing AI agents with information-flow control, arXiv:2505.23643 (2025).

[27] M. Aldinucci, A. Bracciali, I. Colonnelli, D. Medić, A. Mulone, L. Roversi, M. E. Santimaria, Hard-KLAIM: Hardening MCP-enabled agentic workflows with a locality-aware type-and-effect system for trace safety, Journal of Logical and Algebraic Methods in Programming (2026) 101191. In press, journal pre-proof; available online 9 October 2026. doi:10.1016/j.jlamp.2026.101191.

[28] C. Ciliberto, A. Rudi, E. Tassi, Proof-carrying contracts: Commitments AI agents cannot break, arXiv:2610.28484v1 (2026).
URL https://arxiv.org/abs/2610.28484v1

[29] Costwright contributors, Costwright source repository, https://github.com/eleata/costwright, source repository, not peer reviewed; accessed 24 October 2026 (2026).

[30] Typed Resources contributors, Typed resources source repository, https://github.com/eleata/typed-resources, source repository, not peer reviewed; accessed 24 October 2026 (2026).

Appendix A. Historical supplementary implementation

The archive retains `legacy/v1`, an earlier C++/Lean certificate analyzer. It uses a different AST and a policy that also restricts prompt exposure to the model. There is no proved translation to the Python/Lean system studied here, and its test counts are excluded from the main implementation evaluation. Its retained checks cover 106 C++ tests, 82 Python tests and four Lean files. The exact-accounting study records 5,344 texts, 8,996 concatenations and 64 configurations, including 24 unsupported configurations. These finite tokenizer observations help motivate complete-request accounting, not a scalar bound for arbitrary tokenizers.

Its framework replay completes 24/24 cases after a failed environment attempt. Independent repetition differs in 2/24 pairs, whereas captured replay agrees in 24/24. Four compiled semantic mutants are detected. These historical observations are supplementary checks of that independent implementation.

Appendix B. Earlier application cohorts

Earlier evidence remains immutable supplementary material. The original main cohort retained 962 pairs; a later completion added 210 main pairs and

512 Qwen replication pairs under a different implementation. Template/Unicode corrections were checked separately, including a 48-pair workflow study, a 192-case primitive matrix and a 24-pair release check. A targeted comparison differed from older saved fields in 6/18 examples. Those observations are not pooled with the present 1,684-pair homogeneous study. The artifact retains their protocols, raw records, generated tables and implementation identities, together with the historical timing counterexample and executed baseline microbenchmarks.

Appendix C. Reproduction map

Start with `publication/README.md` and its claim ledger of mechanized results, finite observations, assumptions and external actions. The source-policy example gives three concrete checker outcomes:

```
python experiments/source_binding_case.py \
  --output out/policy.json
# candidate-policy: accepted; source-bound unshaped: rejected;
# source-bound repair: accepted
python experiments/admission_domain.py --output out/domain.json
# finite_cases: 45; all_completed_with_static_shape: true
```

Compile the core object before the domain extension. Set `LEAN_PATH=proof` for the second command (PowerShell syntax: README).

```
lean -o proof/ShapeCore.olean proof/ShapeCore.lean
LEAN_PATH=proof lean -o proof/AdmissionDomain.olean \
  proof/AdmissionDomain.lean
LEAN_PATH=proof lean proof/StaticAdmissionChecks.lean
python -m unittest discover -s tests -v
python experiments/differential.py --programs 1000 --runs 4 \
  --output out/differential.json
python experiments/correspondence.py --programs 150 \
  --output out/correspondence
python publication/build_evidence.py --require-complete
```

The operational records retain kernels, stores, candidates and expected outputs. The independent interpreter imports neither compiler nor runtime; the manifest hashes all three implementations. Model tests require the pinned GGUFs; model-free runs report explicit skips.